

DSA & Advanced DSA Roadmap

Phase 17 — Competitive Programming & Advanced Optimization

Learning Objectives

By the end of this phase, learners will be able to:

- Implement and apply Binary Indexed Trees (Fenwick Trees) and Segment Trees for range query and range update problems faster than plain prefix sums allow.
- Apply modular arithmetic and fast (binary) exponentiation to problems involving very large numbers.
- Use the Sieve of Eratosthenes and Euclid's algorithm (GCD) as standard building blocks rather than re-deriving them under time pressure.
- Apply binary lifting to answer ancestor and Lowest Common Ancestor queries on trees in $O(\log n)$ after $O(n \log n)$ preprocessing.
- Recognize when a contest-style problem's constraints (e.g. $n \leq 10^5$ with many queries) signal the need for one of these advanced data structures rather than a straightforward array-based approach.

Range Query Techniques

Prefix Sum, Difference Array, and Range Updates — Review and Extension

Concepts

- Reviewing Phase 1's Prefix Sum and Difference Array patterns as the baseline: $O(1)$ range-sum queries and $O(1)$ range updates, but only when the underlying array does not also need point updates mixed with range queries.
- Identifying the gap those techniques cannot fill: when both point updates and range queries must be fast and interleaved, plain prefix sums require $O(n)$ to rebuild after each update.
- Binary Indexed Tree (Fenwick Tree) — supports both point update and prefix-sum query in $O(\log n)$, filling exactly that gap, using the lowest-set-bit trick to jump between relevant array positions.
- Segment Tree — a more general structure supporting arbitrary range queries (sum, min, max) and range updates in $O(\log n)$, at the cost of more implementation complexity than a BIT.
- Lazy Propagation — deferring a pending range update at a segment tree node until that node is actually visited again, avoiding the $O(n)$ cost of immediately updating every affected leaf.

Practice Problems

- Given an array, support both updating an element's value and querying the sum of a range, repeatedly and efficiently.
- Given an array, count for each element how many later elements are smaller than it, using a Binary Indexed Tree over value ranks.

LeetCode

- [Range Sum Query - Mutable](https://leetcode.com/problems/range-sum-query-mutable) — leetcode.com/problems/range-sum-query-mutable
- [Count of Smaller Numbers After Self](https://leetcode.com/problems/count-of-smaller-numbers-after-self) — leetcode.com/problems/count-of-smaller-numbers-after-self

Note: Count of Smaller Numbers After Self is the exact problem flagged back in Phase 8 as requiring a Binary Indexed Tree; it is assigned here, where that structure is formally taught, rather than in Phase 8.

Advanced Searching

Binary Search on Answer and Parametric Search

Concepts

- Reviewing Phase 7's Binary Search on Answer pattern (Koko Eating Bananas, Capacity to Ship Packages) as the foundation.
- Parametric Search — the general technique name for binary-searching over a continuous or discrete answer space using a monotonic decision predicate, of which Phase 7's problems were specific instances.
- Recognizing parametric search opportunities in optimization problems phrased as 'minimize the maximum' or 'maximize the minimum,' which are common in competitive programming but not always obviously framed as search problems.

No new problems assigned for this category: the implementation and pattern depth was already covered in Phase 7. This entry exists to formally name 'parametric search' as the generalized technique umbrella and to point back to that phase's problems as the worked examples.

Mathematical Techniques

Modular Arithmetic, Fast Exponentiation, GCD, and Sieve

Concepts

- Modular Arithmetic — the rules for performing addition, subtraction, and multiplication under a modulus, and why this is necessary when an exact answer would otherwise overflow standard integer types.
- Fast Exponentiation — computing a^b in $O(\log b)$ by repeated squaring, instead of $O(b)$ repeated multiplication; revisits Phase 0's $\text{Pow}(x, n)$ at a deeper level, now combined with a modulus.
- GCD (Euclid's Algorithm) — computing the greatest common divisor in $O(\log(\min(a, b)))$ via repeated remainder-taking, a building block for simplifying fractions and detecting coprimality.
- Sieve (of Eratosthenes) — precomputing primality (or smallest prime factor) for every number up to n in $O(n \log \log n)$, avoiding per-number primality tests that would cost $O(n\sqrt{n})$ overall.

Practice Problems

- Compute $a^b \bmod m$ efficiently when b is given as an array of digits too large to fit in a standard integer.
- Count the number of prime numbers strictly less than a given n using a sieve rather than per-number trial division.
- Compute the greatest common divisor of two strings, where the GCD of strings 'str1' and 'str2' is the largest string that both can be formed by repeating.

LeetCode

- [Super Pow — leetcode.com/problems/super-pow](https://leetcode.com/problems/super-pow)
- [Count Primes — leetcode.com/problems/count-primes](https://leetcode.com/problems/count-primes)
- [Greatest Common Divisor of Strings — leetcode.com/problems/greatest-common-divisor-of-strings](https://leetcode.com/problems/greatest-common-divisor-of-strings)

Note: $\text{Pow}(x, n)$, already assigned in Phase 0, covered fast exponentiation without a modulus. Super Pow is a deliberately different, harder problem combining fast exponentiation with modular arithmetic on an exponent too large to represent directly.

Advanced Tree Techniques

Euler Tour, Ancestor Queries, and Subtree Queries

Concepts

- Euler Tour — flattening a tree into a linear sequence by recording each node at both its entry and exit time during a DFS, so that subtree-related questions become range-related questions on that flattened array.
- Subtree Queries — using a node's Euler Tour entry/exit time range to identify exactly which array positions correspond to its entire subtree, then answering subtree-aggregate questions (sum, count) using the range-query structures from earlier in this phase.

- Ancestor Queries via Binary Lifting — precomputing each node's 2^0 -th, 2^1 -th, 2^2 -th, ... ancestor, so any k -th ancestor can be found in $O(\log k)$ by combining powers of two, rather than $O(k)$ by walking up one parent at a time.

Practice Problems

- Given a tree (as a parent array) and many queries of the form 'what is the k -th ancestor of this node,' answer each query in $O(\log n)$ after preprocessing.

LeetCode

- [Kth Ancestor of a Tree Node — leetcode.com/problems/kth-ancestor-of-a-tree-node](https://leetcode.com/problems/kth-ancestor-of-a-tree-node)

Note: this formalizes the binary lifting technique that the Phase 11 Lowest Common Ancestor pattern gestured toward but did not require; the LCA problems there were solved with simpler $O(n)$ -per-query recursive approaches, sufficient at that scale.

Advanced Data Structures

Union Find (DSU), Segment Tree, and Lazy Propagation — Summary

Concepts

- Union Find (DSU) was fully covered in Phase 14 (Graphs) with path compression and union by rank; no new treatment is needed here beyond noting it as a foundational advanced data structure that recurs throughout competitive programming.
- Segment Tree and Lazy Propagation were introduced earlier in this phase under Range Query Techniques; this entry exists to name them explicitly as the 'Advanced Data Structures' the original curriculum outline calls for, consolidating rather than duplicating that material.

Phase Assessment

Implementation Tasks

- Build a Binary Indexed Tree from scratch supporting point update and prefix-sum query, then use it to solve Count of Smaller Numbers After Self.
- Build a Segment Tree from scratch supporting range-sum query and point update, then extend it with lazy propagation to support range updates without per-leaf cost.
- Implement binary lifting from scratch for the Kth Ancestor of a Tree Node problem, and verify it answers correctly even on a deliberately unbalanced (skewed) tree.

Recommended LeetCode Target

Medium Problems

Range Query Techniques

- leetcode.com/problems/range-sum-query-mutable
- leetcode.com/problems/super-pow
- leetcode.com/problems/greatest-common-divisor-of-strings

Hard Problems

- leetcode.com/problems/count-of-smaller-numbers-after-self
- leetcode.com/problems/kth-ancestor-of-a-tree-node

Count Primes (leetcode.com/problems/count-primes) is Easy difficulty and listed under Mathematical Techniques above.

On scope and dedup

This phase has 6 distinct problems across 5 technique categories. Two categories (Advanced Searching, Advanced Data Structures — specifically Union Find) are deliberately problem-free here: their teaching material and problems were already delivered in full in Phase 7 and Phase 14 respectively, and this phase formally names and cross-references those techniques rather than re-teaching or re-assigning them. None of the 6 problems overlaps with any problem used in Phases 0–16, including the close cousins Pow(x, n) (Phase 0) and the Phase 11 LCA problems.

Coding Challenges (Assessment Day)

- Range Sum Query - Mutable (Segment Tree or BIT)
- Count of Smaller Numbers After Self
- Super Pow
- Kth Ancestor of a Tree Node

Expected Outcome

Upon completing Phase 17, and the roadmap as a whole, students will:

- Understand data structures beyond API usage, having implemented every structure in this roadmap from scratch at least once.
- Implement core structures from scratch, including the advanced ones introduced in this final phase: Binary Indexed Trees, Segment Trees with lazy propagation, and binary lifting.
- Recognize and apply interview and contest patterns across all 17 phases, from array traversal through advanced tree and range-query techniques.
- Solve Easy, Medium, and selected Hard LeetCode problems, drawn from a roadmap-wide pool of problems with zero duplication, verified individually for correctness and accessibility.
- Perform well in service-company placements, compete confidently in product-company coding assessments, and approach competitive programming contests with a structured toolkit rather than ad hoc pattern-matching.